



Jour 1 : Les différents types d'apprentissage en Machine Learning

Bien le bonjour, et bienvenue !

Cette semaine, on ne va pas juste "voir" le Machine Learning de loin. On va le pratiquer, le casser, le comparer, et finir vendredi avec quelque chose de concret sur votre GitHub.

Le fil rouge de la semaine, c'est **L'Arène des Algos**. L'idée : à chaque fois qu'on apprend un nouvel algorithme, on le fait entrer dans une arène et on le compare aux autres sur un même problème. Qui prédit le mieux ? Qui est le plus rapide ? Qui se trompe le moins sur les cas tordus ? À la fin de la semaine, vous saurez non seulement faire tourner un modèle, mais surtout choisir le bon et justifier votre choix. C'est exactement ça, le métier.

Aujourd'hui, jour 1, on plante le décor : c'est quoi le Machine Learning, à quoi ça sert vraiment en entreprise, comment on structure un projet, et surtout on monte ensemble votre tout premier pipeline qui apprend et qui prédit. Pas de panique si certains mots vous sont inconnus : on les définit au fur et à mesure. Let's go.

Accueil et Présentation

Avant de démarrer la moindre notion, je vais vous dire qui je suis, et surtout j'ai envie de savoir qui vous êtes. Quelques questions pour calibrer la journée.

Qui est dans la salle ?

*Qui a déjà entendu parler de Machine Learning ailleurs que dans une pub ou un article obscure sur l'IA ?
(en fait, qui a déjà vu ça dans une pub, perso pas moi)*

Qui en a déjà fait, ne serait-ce qu'un tuto, un cours en ligne, un projet perso ?

Et côté outils, juste pour situer :

- Votre niveau en Python : à l'aise au quotidien, ou ça date un peu ? Qu'est-ce que vous avez pu créer ?
- pandas, vous connaissez ? Vous l'avez utilisé à quel point ? C'est la bibliothèque pour manipuler des tableaux de données en Python. On va s'en servir tous les jours.
- Quelqu'un a déjà touché à scikit-learn, TensorFlow, PyTorch ? (Si non, c'est parfait, c'est le but de la semaine.)
- Votre setup : VS Code, Jupyter en local, Google Colab ?

Y a-t-il quelqu'un qui a déjà fait un stage en data, ou un projet perso "sérieux" ?

Calibrage rapide

Parmi ces termes, lesquels vous paraissent flous ? (Aucun problème, c'est juste pour savoir où on met le curseur).

moyenne, médiane, écart-type, corrélation, matrice, pandas, matplotlib, lire de la doc en anglais.

Aucun de ces mots n'est un prérequis bloquant. On les recroisera, et je les réexpliquerai au moment où on en a besoin.

Le fil rouge et le deal

En 5 jours, vous allez accumuler des compétences qui se déposent dans un repo GitHub : à la fin, vous avez une vraie pièce de portfolio data.

- **Aujourd'hui** : le paysage du ML, et votre premier pipeline qui apprend.
- **J2** : nettoyer une donnée sale. Sans ça, le meilleur algo du monde donne de la bouillie.
- **J3** : l'arsenal complet des algos, et le grand combat de l'Arène.
- **J4** : on découvre les réseaux de neurones, on apprend à juger un modèle proprement, et on le déploie.
- **J5** : projet en groupe, soutenance, démo live.

Je ne le dirai jamais assez, **coupez-moi la parole !!!** Le but n'est pas de m'écouter faire un monologue, et on n'est pas sensé tout connaître à l'avance (sinon mon cours sert à rien). Donc aucune question n'est bête ! Si vous attendez la fin de ma phrase pour poser une question, c'est déjà trop tard.

Niveau méthodologie : je fais toujours au mieux pour qu'on équilibre théorie et pratique (qu'on ne se prenne pas des heures de théorie dans la figure à en finir assomé). Je montre / je vous fais découvrir et hop vous mettez les mains dans le cambouis.

Planning de la Journée

1. **Le paysage** : Data Science, IA, ML, DL, qui est qui et où ça se place dans la chaîne de la donnée
2. **Petite histoire** : d'où vient le ML, et pourquoi le classique reste roi
3. **Où ça sert vraiment** : cas d'usage en entreprise, et la communauté (Kaggle)
4. **Comment on mène un projet data** : la méthode CRISP-DM
5. **Les familles d'apprentissage** : supervisé, non-supervisé, et les autres
6. **Les outils** : panorama des frameworks, et pourquoi scikit-learn pour nous
7. **Votre premier pipeline** : charger, entraîner, prédire, mesurer
8. **Limites, biais et responsabilité** : un modèle n'est ni neutre ni infaillible
9. **L'après-midi (À votre tour)** : monter un pipeline complet sur un vrai dataset et le faire entrer dans l'Arène

Chaque notion du matin suit le même rythme : je pose le concept et je montre le code complet, vous jouez dedans en changeant un paramètre, on confronte avec le voisin. Puis on enchaîne.

1 - Le paysage : IA, ML, DL, Data Science

Imaginons qu'on regarde Netflix. La plateforme nous recommande une série dont on n'a jamais entendu parler, et contre toute attente on adore. Mais du coup, est-ce que c'est de l'IA ? Du ML ? Du Deep Learning ? La réponse compte, parce que selon le cas on n'utilise pas les mêmes outils, ni les mêmes données, ni les mêmes méthodes !

L'essentiel

Quatre mots reviennent en boucle dans le métier, et on les emploie souvent de travers. Mettons de l'ordre.

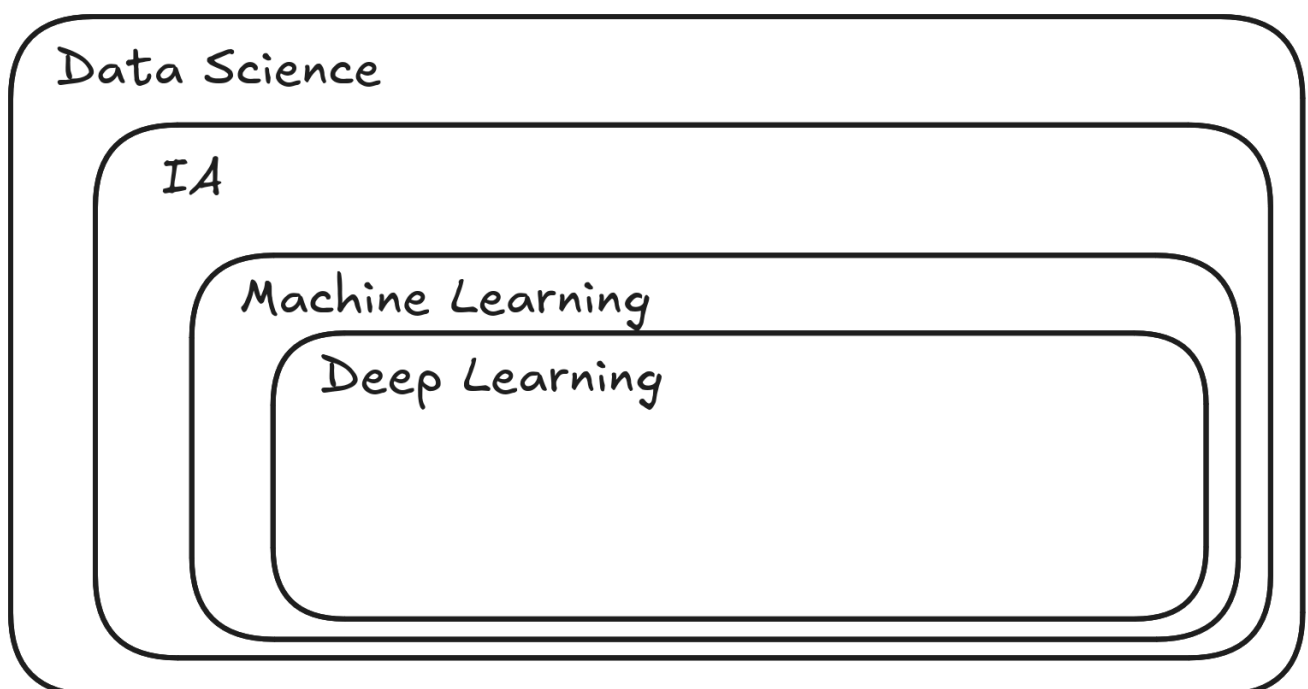
Instant Notions : le vocabulaire du domaine

- **Data Science** : la discipline globale qui consiste à tirer de la valeur de la donnée. Ça inclut la collecte, le nettoyage, la visualisation, les statistiques, et la modélisation. Un data scientist passe une grande partie de son temps sur la donnée elle-même, pas sur l'algo.
- **IA (Intelligence Artificielle)** : le terme parapluie. Toute technique qui permet à une machine d'imiter un comportement intelligent. Un moteur de règles "si le client achète X, recommander Y" est déjà de l'IA, même sans aucun apprentissage.
- **ML (Machine Learning)** : sous-domaine de l'IA où la machine apprend par l'exemple plutôt que par des règles écrites à la main. On lui montre des milliers de paires (entrée, sortie attendue), et elle trouve elle-même les motifs.
- **DL (Deep Learning)** : sous-domaine du ML qui utilise des réseaux de neurones à plusieurs couches. Les pixels d'une image deviennent des bords, puis des formes, puis des visages.

Ok mais quelle différence entre data science et data analyst ?

=> Le Data Analyst explore et interprète des données existantes pour répondre à des questions business précises, tandis que le Data Scientist va plus loin en construisant des modèles prédictifs et des algorithmes de machine learning pour anticiper des tendances ou automatiser des décisions.

Pour bien visualiser les choses,



La relation : Data Science est la pratique englobante => à l'intérieur, l'IA => dans l'IA, le ML => et dans le ML, le DL.

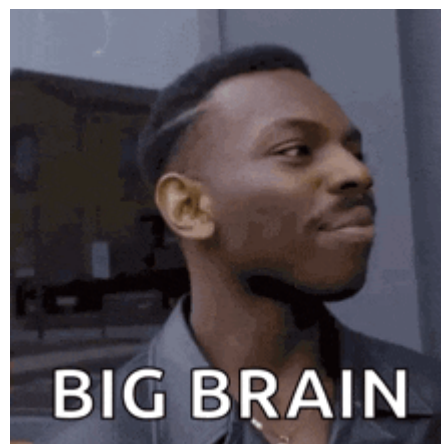
Ce cours porte sur le ML (le coeur, J1 à J3), et on ouvre la porte du DL en J4.

Où ça se place dans la chaîne de la donnée ? Un projet data, ça ressemble toujours à ça : on collecte la donnée, on la stocke, on la nettoie, on l'analyse, on modélise, on déploie.

Tout le monde dit à peu près la même chose :



Le ML, c'est l'étape de modélisation, presque à la fin. Si les étapes d'avant sont bâclées (donnée sale, mal comprise), le ML ne rattrape rien. D'où le J2 entier consacré à la préparation de la donnée.



Le lien avec les maths et les stats. Le ML n'est pas tombé du ciel : c'est l'enfant de trois disciplines, les **statistiques**, les **mathématiques** et le **data mining**. Pas de panique, on ne fait pas un cours de maths aujourd'hui (les vraies formules arrivent au moment où on en a besoin, surtout en J4). Mais voilà concrètement ce qui se cache derrière les mots, pour qu'ils ne vous surprennent pas quand ils tomberont.

Les maths qu'on va croiser :

- **Vecteur et matrice.** Un vecteur, c'est juste une liste de nombres : les mesures d'un exemple, par exemple `[surface, prix, âge]`. Une matrice, c'est un tableau de nombres : tout votre dataset, une ligne par exemple et une colonne par variable. Tout le ML manipule des matrices. (Partout, formalisé en J4.)
- **Produit scalaire.** Multiplier deux listes terme à terme puis additionner le tout. C'est l'opération de base d'un neurone (pondérer des entrées) et une façon de mesurer si deux vecteurs se ressemblent. (J4.)
- **Dérivée et gradient.** La dérivée mesure "dans quel sens et à quelle vitesse ça monte". Le gradient, c'est la même idée mais en plusieurs dimensions : la direction de plus forte pente. (Le coeur de l'apprentissage d'un réseau, J4.)
- **Descente de gradient.** Pour réduire son erreur, le modèle descend la pente petit à petit, comme un randonneur qui cherche le fond de la vallée dans le brouillard, un pas après l'autre. (Comment un modèle "apprend", J4.)
- **Distance euclidienne.** La distance "à vol d'oiseau" entre deux points (le théorème de Pythagore, généralisé). Sert à dire si deux exemples sont proches. (KMeans, J3.)

Les stats qu'on va croiser :

- **Moyenne, médiane, écart-type.** La moyenne est le centre ; la médiane est la valeur du milieu, plus robuste aux extrêmes (un seul salaire de PDG fait exploser la moyenne, pas la médiane) ; l'écart-type mesure à quel point les valeurs sont dispersées autour du centre. (Décrire et nettoyer une colonne, repérer les outliers, J2.)
- **Corrélation.** Un nombre entre -1 et 1 qui dit si deux variables bougent ensemble : proche de 1, quand l'une monte l'autre monte ; proche de -1, c'est l'inverse ; proche de 0, aucun lien. Piège à retenir : corrélation n'est pas causalité. (Repérer les variables liées et la multicolinéarité, J2.)
- **Probabilité et loi de Bayes.** Une probabilité quantifie une incertitude (un nombre entre 0 et 1). La loi de Bayes met à jour une croyance quand une nouvelle info arrive. (La régression logistique sort une proba, Naive Bayes est entièrement bâti là-dessus, J3.)
- **Statistiques inférentielles.** Tirer une conclusion sur un tout (la population) à partir d'un échantillon : estimer la satisfaction de tous vos clients en n'en interrogeant que 1000. C'est LA raison pour laquelle on teste un modèle sur des données qu'il n'a jamais vues : on veut savoir s'il généralise, pas s'il a appris par coeur. (Split train/test, validation croisée, J4.)

Et le data mining ? C'est l'exploration de gros volumes de données pour y dénicher des motifs utiles : des associations, des groupes, des anomalies. Historiquement, c'est la discipline d'où vient une bonne partie des algos qu'on verra. L'esprit : fouiller la donnée jusqu'à ce qu'elle parle.

Au fond, quand un algo "apprend", il fait de la statistique appliquée à grande échelle, portée par un peu d'algèbre et de calcul. Voilà tout le secret.

Pour creuser : [3Blue1Brown](#), la playlist *Essence of Linear Algebra* pour voir vecteurs et matrices en images, et [StatQuest](#), la playlist *Statistics Fundamentals* pour les stats de base. Une heure chacune, et tout le cours devient plus limpide.

Bougez un paramètre : Pour chacun des exemples ci-dessous, c'est plutôt IA-sans-apprentissage, ML, ou DL ?

- un thermostat qui déclenche le chauffage en dessous de 19 degrés
- un filtre anti-spam qui s'améliore avec vos signalements
- une app qui reconnaît la race d'un chien sur une photo

On est tous à peu près d'accord sur les trois ?

(Indice : le premier n'apprend rien, le deuxième apprend de données étiquetées, le troisième manipule des pixels.)

Pour aller plus loin

Là où ça devient intéressant, c'est que ces frontières bougent. La reconnaissance vocale de votre téléphone : DL. Les filtres de Snapchat qui détectent les visages : DL. La recommandation Spotify qui devine votre humeur à 23h : un mélange de ML classique et de DL. Beaucoup de systèmes réels combinent plusieurs approches.

Il y en a qui sont là depuis "toujours" et qu'on ne remarque jamais, comme la suggestion de mot suivant dans votre smartphone.

Et un piège classique d'entretien : "le Deep Learning, c'est toujours mieux que le ML classique ?" Non. Sur des données tabulaires (des tableaux de chiffres, comme un export Excel), un bon algo classique comme un Random Forest bat très souvent un réseau de neurones, tout en étant plus rapide à entraîner et plus facile à expliquer. C'est précisément pour ça que ce cours met le ML classique au centre.

Le vocabulaire de la semaine

On va employer une dizaine de mots non-stop pendant cinq jours. Autant les poser maintenant : je vous laisse cette liste sous le coude, c'est votre ptit dico de survie. Pas besoin de tout mémoriser, on les répétera tellement qu'ils deviendront naturels.

- **Feature (variable, caractéristique)** : une colonne de votre tableau, une information en entrée. La surface d'un appartement, l'âge d'un client. Un modèle apprend à partir des features.
- **Label (étiquette, cible, target)** : la réponse à prédire. Le prix de l'appart, "spam ou pas spam". En supervisé, chaque exemple a son label.
- **Sample (exemple, observation, instance)** : une ligne de votre tableau, un cas concret. Un appartement, un client, un email.
- **Dataset** : l'ensemble des samples. Votre tableau complet.
- **Modèle** : l'objet qui, une fois entraîné, transforme des features en prédiction. C'est ce que `.fit()` fabrique.
- **Entraînement (training, fit)** : la phase où le modèle apprend les motifs sur les données.
- **Inférence (prédiction, predict)** : la phase où le modèle entraîné répond sur de nouvelles données.
- **Hyperparamètre** : un réglage qu'on choisit AVANT l'entraînement (le nombre d'arbres d'une forêt, la profondeur max d'un arbre). À ne pas confondre avec les **paramètres**, que le modèle, lui, apprend tout seul.
- **Généralisation** : la capacité du modèle à bien marcher sur des données jamais vues. C'est le but ultime. Un modèle qui récite par coeur ne généralise pas.
- **Overfitting (surapprentissage)** : le modèle apprend par coeur le jeu d'entraînement, bruit compris, et se plante sur du neuf. Comme un élève qui mémorise les annales sans rien comprendre.
- **Underfitting (sous-apprentissage)** : le modèle est trop simple, il rate même les motifs évidents. L'élève qui n'a pas assez révisé.
- **Biais et variance** : les deux façons de se tromper. Trop de biais = underfitting (modèle trop rigide). Trop de variance = overfitting (modèle trop sensible aux détails). Tout l'art du ML, c'est de trouver l'équilibre. On le verra concrètement en J3 et J4.

Aparté : ne confondez pas "biais" au sens biais/variance (un défaut technique du modèle) et "biais" au sens des biais de données de la section 8 (un problème éthique). Même mot, mais deux sens, le contexte tranche.

Les métiers de la data

"Data scientist" est devenu un mot-valise. Dans une vraie équipe, plusieurs rôles se partagent le travail, et savoir qui fait quoi vous aide à vous situer (et à viser le bon poste).

- **Data Analyst** : explore et visualise la donnée existante, répond à des questions business ("quelles régions vendent le plus ?"). Outils : SQL, Excel, un peu de Python, des dashboards (Power BI, Tableau).
- **Data Scientist** : construit des modèles prédictifs. C'est le coeur de ce cours. Outils : Python, scikit-learn, pandas.
- **Data Engineer** : construit les "tuyaux" qui collectent, nettoient et stockent la donnée à grande échelle (les pipelines de données, à ne pas confondre avec le pipeline de modélisation). Outils : SQL, Spark, cloud.
- **ML Engineer / MLOps** : met les modèles en production, les surveille, gère leur cycle de vie. À cheval entre data science et dev. On en touche un bout en J4.

La frontière est floue, surtout en petite boîte où une seule personne fait tout. Mais en entretien, savoir nommer ces rôles montre que vous comprenez l'écosystème.

Bon à savoir : dans la majorité des offres "data scientist junior", on attend surtout que vous sachiez le contenu de cette semaine : préparer une donnée, entraîner et comparer des modèles, expliquer vos choix. Pas besoin d'un doctorat pour démarrer.

Et l'IA générative dans tout ça ?

Vous vous demandez sûrement où ChatGPT, Midjourney et compagnie se placent. Réponse : c'est du Deep Learning, branche "générative".

- **IA discriminative** (ce qu'on fait surtout cette semaine) : on prédit une étiquette ou une valeur. "Cet email est-il un spam ?".
- **IA générative** : on produit du contenu neuf, texte, image, son. "Écris-moi un email", "dessine un chat astronaute".

Les modèles de langage (LLM / Large Language Model) comme ChatGPT (ou Claude, ou Gemini...) sont des réseaux de neurones géants (des Transformers, 2017) entraînés à prédire le mot suivant sur une montagne de texte. Les **agents** vont un cran plus loin : un LLM qui peut utiliser des outils (chercher sur le web, exécuter du code) pour accomplir des tâches en plusieurs étapes.

Tout ça, c'est passionnant et c'est l'actualité, mais c'est au bout du chemin. Pour vraiment comprendre un Transformer, il faut d'abord comprendre un neurone, une couche, un gradient. C'est exactement ce qu'on bâtit cette semaine. Voyez ce cours comme les fondations : sans elles, l'IA générative reste une boîte magique qu'on subit au lieu de la piloter.

Pour creuser : si le sujet vous démange, [3Blue1Brown](#) a une série en images sur les réseaux de neurones et les Transformers. À garder pour la fin de semaine, quand les bases seront en place.

On a posé le décor et le vocabulaire. Petit détour avant d'aller plus loin : d'où vient tout ça ? Un peu d'histoire, parce que ça explique pourquoi on travaille comme on travaille aujourd'hui.

2 - Petite histoire : d'où vient tout ça

Avant d'aller voir où le ML atterrit, un détour qui vaut le coup : le ML n'est pas né avec ChatGPT, loin de là. Connaître les grandes dates, ça aide à comprendre pourquoi on travaille comme on travaille, et surtout pourquoi le ML classique n'est pas un truc de papy.

L'essentiel

Les moments qui ont compté :

- **1958 : le Perceptron.** Frank Rosenblatt invente le premier "neurone artificiel" qui apprend. Le New York Times annonce que la machine va bientôt marcher, parler et se reproduire. (Spoiler : pas tout de suite.)
- **Les hivers de l'IA (années 70-80).** Trop de promesses, pas assez de résultats ni de puissance de calcul. Les financements s'effondrent. Deux fois.
- **1984 : les arbres de décision (CART).** Breiman et ses collègues posent les arbres, l'ancêtre direct du Random Forest. Du ML lisible par un humain.
- **1986 : la backpropagation.** Rumelhart, Hinton et Williams popularisent l'algo qui permet d'entraîner des réseaux à plusieurs couches. On recroisera ce mot en J4.
- **Années 90 : les SVM.** Vapnik et Cortes. Pendant quinze ans, la star de la classification. On les verra en J3.
- **2001 : le Random Forest.** Breiman encore. Robuste, simple à régler, redoutable sur le tabulaire. Toujours un favori en 2025.
- **2012 : le moment AlexNet.** Un réseau profond explose la compétition de reconnaissance d'images ImageNet. C'est LE déclencheur du boom Deep Learning moderne.
- **2014 : XGBoost.** Le gradient boosting qui enchaîne les victoires sur Kaggle, surtout sur données tabulaires. On le croise en J3.
- **2017 : les Transformers.** Le papier "Attention is all you need", l'architecture derrière tous les modèles de langage actuels.
- **2022 : ChatGPT.** Le ML débarque dans le quotidien du grand public.

C'est quoi un "hiver de l'IA" ? Une période où les promesses non tenues assèchent les financements et l'intérêt pour l'IA. Il y en a eu deux. Bon à garder en tête quand vous lisez les annonces tonitruantes d'aujourd'hui.

Un truc à retenir : entre 2001 et 2014, les algos qui dominaient n'étaient pas des réseaux de neurones, c'étaient des forêts et du boosting. Et sur les données tabulaires d'entreprise, c'est encore largement vrai aujourd'hui. Voilà pourquoi ce cours passe trois jours sur le ML classique avant d'ouvrir la porte du DL.

Mais les forêts c'est quoi ? (moi je connais que celles avec les arbres en bois)

Mhh cool tu connais déjà les arbres en bois. ~~Pas besoin d'en savoir plus.~~ Le Random Forest, c'est pareil : une forêt d'arbres de décision. L'idée : plutôt que de se fier à un seul arbre (qui peut se tromper), on en entraîne des centaines, chacun sur un sous-ensemble aléatoire des données. Ensuite, chaque arbre vote, et c'est la majorité qui gagne. Un arbre seul est fragile... mais une forêt entière, beaucoup plus robuste



Et... **Le boosting ?**

Le XGBoost c'est une autre façon d'assembler des arbres, mais avec une logique différente : au lieu de les entraîner en parallèle et de voter, on les entraîne en séquence. Chaque nouvel arbre se concentre sur les erreurs du précédent, pour les corriger. Au final, on additionne les contributions de tous les arbres. C'est plus précis, mais aussi plus sensible au surapprentissage si on ne règle pas bien les paramètres.

On pousse plus loin

Un point de culture qui fait son effet en entretien : beaucoup d'idées "modernes" sont vieilles. La backpropagation (1986) attendait juste deux choses pour exploser : assez de données, et des cartes graphiques (GPU) assez puissantes. Les deux sont arrivés vers 2010. Autrement dit, ce n'est pas la théorie qui a débloqué le Deep Learning, c'est le matériel et la donnée. Une leçon qui se répète souvent : l'idée existe bien avant que le monde soit prêt à la faire tourner.

Pour creuser : [StatQuest](#) pour l'intuition de chaque algo, [Two Minute Papers](#) pour suivre l'actu de la recherche.

Les épisodes qu'on oublie souvent

La frise du dessus va à l'essentiel. Mais quelques épisodes méritent leur minute, parce qu'ils racontent comment le domaine a vraiment avancé : par à-coups, pas en ligne droite.

- **Les systèmes experts (années 80).** Avant le ML, on a cru que l'IA, ce serait des milliers de règles "si... alors..." écrites à la main par des experts. Ça a marché un peu, puis ça s'est effondré sous son propre poids : impossible de maintenir des dizaines de milliers de règles qui finissent par se contredire. La leçon : apprendre des données bat coder des règles à la main, dès que le problème devient complexe.
- **1997 : Deep Blue bat Kasparov aux échecs.** Énorme symbole, mais attention : Deep Blue ne faisait quasiment pas de ML. C'était de la force brute (explorer des millions de coups) plus des règles d'experts. L'IA qui "gagne" n'est pas toujours l'IA qui "apprend".
- **2011 : Watson gagne à Jeopardy!.** IBM montre qu'une machine peut comprendre des questions en langage naturel et fouiller une masse de connaissances. Le début de la grande hype "l'IA va tout changer".
- **2016 : AlphaGo bat Lee Sedol au Go.** Là, c'est du vrai ML (apprentissage par renforcement plus réseaux profonds). Le Go était réputé "impossible" pour une machine avant des décennies. AlphaGo a joué des coups que des siècles de maîtres humains n'avaient jamais imaginés. Moment de bascule. [Super documentaire](#) dessus si vous voulez
- **2012-2020 : l'effet GPU.** Les cartes graphiques, conçues pour les jeux vidéo, se révèlent parfaites pour les calculs des réseaux de neurones (plein de petites opérations en parallèle). Nvidia, vendeur de cartes pour gamers, devient un géant de l'IA. Sans ce hasard matériel, pas de boom DL.
- **L'hypothèse de l'échelle (le "scaling").** La grande découverte des années 2020 : souvent, pour faire mieux, il "suffit" de plus de données, plus de paramètres, plus de calcul. C'est ce qui a donné GPT. Mais ça coûte des millions en électricité et en cartes, et ça ne marche pas sur tous les problèmes, surtout pas sur vos données tabulaires d'entreprise.

Le fil rouge de toute cette histoire : les grands sauts viennent autant du matériel et de la donnée que des idées. Et à chaque emballement médiatique, le ML classique continue à résoudre la grande majorité des problèmes réels.

On sait d'où ça vient. Voyons maintenant où ça atterrit : dans quelles boîtes, sur quels problèmes, et où les pros s'affrontent.

3 - Où ça sert vraiment

Le ML, ce n'est pas un gadget de labo. Chaque secteur l'utilise déjà, souvent sans que le grand public le voie. Avant de coder, il faut sentir les cas d'usage : c'est ce qui vous permettra, en J5 et en entreprise, de reconnaître un problème "modélisable".

L'essentiel

Quelques cas concrets, secteur par secteur. Pour chacun : la donnée, la famille d'algo, et l'enjeu réel.

- **Banque, détection de fraude.** Sur des millions de transactions par jour, à peine 0,1% sont frauduleuses : les données sont donc ultra-déséquilibrées (on y reviendra, ça casse les modèles naïfs). Le modèle note chaque transaction en quelques millisecondes et bloque celles qui sentent l'arnaque. L'enjeu est asymétrique : rater une vraie fraude coûte de l'argent à la banque, mais bloquer un client honnête en pleine caisse le rend furieux. C'est de la classification supervisée.
- **Banque, scoring de crédit.** Une banque veut estimer le risque qu'un emprunteur ne rembourse pas avant de lui accorder un prêt. Le modèle apprend sur l'historique financier des anciens clients pour prédire ce risque. Contrainte forte : il doit être **explicable**, parce que le régulateur et le client ont le droit de savoir pourquoi un crédit est refusé. C'est pour ça qu'on préfère souvent une régression logistique ou un arbre, lisibles, à une boîte noire ultra-performante.
- **Assurance.** Une compagnie d'assurance s'en sert pour fixer le prix d'un contrat selon le profil de risque du client (un jeune conducteur paie plus cher qu'un conducteur chevronné), et pour repérer les déclarations de sinistre douteuses. Comme en banque, le modèle doit rester explicable et la fraude est rare, donc les données sont déséquilibrées.
- **Télécom, le churn.** Un opérateur veut savoir quels clients vont résilier le mois prochain, pour les rappeler et les retenir avant qu'ils partent. Le modèle apprend sur la consommation, les appels au support et l'ancienneté pour repérer les signaux de départ. La logique business : retenir un client coûte bien moins cher que d'en reconquérir un. C'est de la classification.

Le churn ? C'est le pourcentage de clients que perd une entreprise sur une période donnée, parce qu'ils arrêtent d'utiliser un service ([source](#))

- **E-commerce et streaming, la recommandation.** Quand Netflix vous propose une série ou qu'Amazon vous suggère un produit, c'est qu'un modèle a appris à partir de ce que des millions d'utilisateurs ont aimé, à deviner ce qui va vous plaire à vous. C'est un mélange de ML classique et de Deep Learning.
- **Santé.** Un modèle peut aider un médecin à repérer une tumeur sur une radio ou à interpréter une analyse sanguine. L'enjeu est énorme : rater une maladie (un faux négatif) peut être dramatique, donc on ne juge surtout pas le modèle sur sa seule accuracy. On creusera ce point en J4.
- **NLP (Natural Language Processing), le texte.** Tout ce qui touche au langage écrit : filtrer les spams dans une boîte mail, deviner si un avis client est positif ou négatif (c'est l'analyse de sentiment), ou faire répondre un chatbot. La machine apprend en quelque sorte à "lire" du texte.
- **Jeux vidéo.** Le ML pilote le comportement des personnages non-joueurs (les PNJ), détecte les tricheurs en repérant les parties au comportement anormal, et ajuste automatiquement la difficulté pour que le joueur ne s'ennuie ni ne décroche.

C'est quoi le NLP ? "Natural Language Processing", le traitement automatique du langage. Tout ce qui consiste à faire lire, comprendre ou générer du texte par une machine : spam, traduction, analyse d'avis, ChatGPT.

Trois exemples historiques valent le détour parce qu'ils ont popularisé le domaine : les **moteurs de recherche** (classer des milliards de pages par pertinence), la **détection de spam** dans les boîtes mail (un des premiers succès grand public du ML, dès les années 2000), et la **lecture automatique de chèques** (reconnaître des chiffres manuscrits, l'un des tout premiers usages industriels des réseaux de neurones).

La communauté et les challenges. Le ML a une particularité géniale (comme beaucoup d'autres secteurs tech) : il y a des compétitions publiques où le monde entier s'affronte sur les mêmes données. La plateforme phare, c'est [Kaggle](#). Des entreprises y publient un problème et un jeu de données, et des milliers de participants soumettent leurs modèles. Netflix a lancé l'une des plus célèbres (le "Netflix Prize", 1 million de dollars pour améliorer leur reco de 10%). Google, et beaucoup d'autres, organisent régulièrement des challenges.

Touchez le terrain : allez sur Kaggle, ouvrez une compétition "Getting Started" (par exemple celle sur le Titanic). Regardez : quelle est la donnée fournie ? Qu'est-ce qu'on doit prédire ? Est-ce une catégorie (survie oui/non) ou une valeur chiffrée ? Gardez votre réponse écrite sous le coude, on s'en resservira dans la section 5.

Vous avez identifié le même type de problème que votre voisin ?

Creusons

Petite note de réalité pro, parce que ça motive. En 2025 en France, un profil data junior (data analyst, junior data scientist) démarre souvent autour de 35 000 à 45 000 euros brut annuels, et ça monte vite avec l'expérience et la spécialisation. Ce qui fait la différence sur un CV débutant : un repo GitHub qui montre de vrais projets, pas juste des tutos recopiés. Et c'est notre but cette semaine !

 **Bonus perso :** le moteur de reco que vous subissez tous les soirs sur Netflix ou Spotify, au fond, c'est un problème de ML que vous saurez poser à la fin de la semaine. Petit défi hors-cours, juste pour le plaisir : notez 15 films que vous avez aimés et 15 que vous avez détestés. C'est, en miniature, un jeu de données étiqueté pour un recommandeur. On verra en J3 comment une machine apprendrait dessus.

 **Des graines pour plus tard :** la plupart de ces cas sont à votre portée d'ici vendredi avec ce qu'on va apprendre. Churn, scoring de crédit, détection de spam, prédiction de prix : ce sont des projets J5 tout à fait possibles, et d'excellents projets perso à mettre sur le portfolio. Repérez dès maintenant celui qui vous parle.

Encore plus de terrains

Le ML s'est glissé partout. Quelques secteurs de plus, pour que vous tapiez dans le mille quand un client ou un recruteur vous parlera de son problème :

- **Retail.** Prévoir la demande pour ne pas être en rupture (ni en surstock), placer les produits, optimiser les prix en temps réel pendant les soldes.
- **Transport et logistique.** Optimiser les tournées de livraison, prédire les retards, anticiper la maintenance d'une flotte avant la panne (la "maintenance prédictive").
- **Énergie.** Prédire la consommation pour équilibrer le réseau, détecter les anomalies sur des capteurs, optimiser le rendement d'éoliennes.

- **Cybersécurité.** Repérer une intrusion ou un comportement anormal dans des millions de logs (de la détection d'anomalie, souvent non-supervisée).
- **Ressources humaines.** Tri de CV (avec tous les risques de biais qu'on verra en section 8), prédiction de l'attrition (le churn, version employés).
- **Agriculture, sport, immobilier, médias...** dès qu'il y a de la donnée et une décision à prendre, il y a un cas d'usage. Le réflexe à entraîner : "quelle est la décision, et quelle donnée pourrait la guider ?".

Bon à savoir : la "maintenance prédictive" est un des cas les plus rentables du ML industriel. Remplacer une pièce juste avant qu'elle lâche, plutôt qu'à date fixe ou après la panne, économise des fortunes. C'est de la régression (combien de temps avant la panne ?) ou de la classification (va-t-elle tomber dans les 30 jours ?).

L'histoire du Netflix Prize

Petite légende qui résume bien la culture du domaine. En 2006, Netflix met 1 million de dollars sur la table : améliorez de 10% la précision de notre système de recommandation. Des milliers d'équipes du monde entier s'affrontent pendant trois ans sur le même jeu de données, un classement public en temps réel.

Trois leçons qui valent pour vous cette semaine :

- **Les gagnants ont gagné en combinant des modèles.** Aucun algo seul ne suffisait : l'équipe victorieuse a mélangé des dizaines de modèles (un "ensemble"). Le tout bat chaque partie. C'est exactement l'esprit de l'Arène : on compare, puis souvent on combine.
- **Netflix n'a jamais déployé le modèle gagnant.** Trop complexe, trop lourd à maintenir en production pour un gain réel modeste. Leçon en or : le meilleur score sur un classement n'est pas toujours le meilleur modèle dans la vraie vie. La simplicité et la maintenabilité comptent.
- **Le format a créé une culture.** Ce concours a inspiré Kaggle et toute la culture du benchmark public. Quand vous comparez vos algos cet après-midi, vous rejouez, en miniature, le Netflix Prize.

Pour creuser : sur [Kaggle](#), ouvrez une compétition et regardez le "Leaderboard" et les "Code" (anciennement kernels) que les gens partagent. Voir comment les autres s'y prennent est l'une des façons les plus rapides de progresser.

On sait pourquoi le ML compte et où les pros se mesurent. Avant de coder dans tous les sens, il faut un cadre : comment mène-t-on un projet data du début à la fin sans se perdre ?

4 - Comment on mène un projet data : CRISP-DM

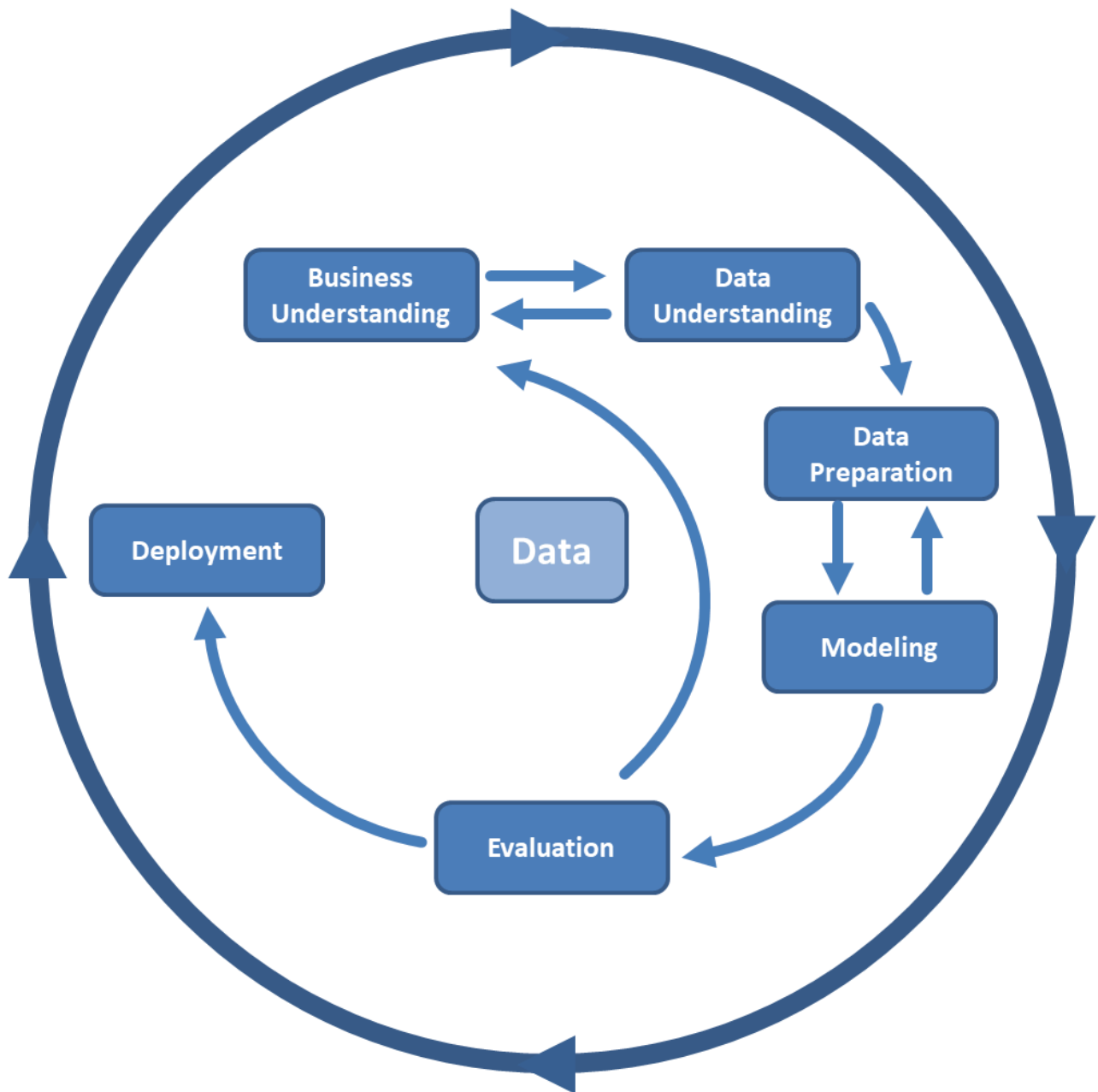
Maintenant, j'ai une question piège (spoiler alert je donne la réponse): quelle est la première erreur d'un débutant en ML ?

Clairement, foncer sur l'algo. On télécharge un dataset, on lance un modèle, et on se retrouve avec un résultat qu'on ne sait ni expliquer ni utiliser, parce qu'on n'a jamais défini ce qu'on cherchait. Les pros suivent une méthode pour éviter ça.

Un peu comme quand on vient d'apprendre à coder, on a une idée, on se lance dessus alors qu'on a même pas encore bien défini l'archi souhaitée, les fonctionnalités, les hors-scope...

L'essentiel

La méthode de référence du secteur s'appelle [CRISP-DM](#) (Cross Industry Standard Process for Data Mining). Ce n'est pas une recette magique, c'est un cadre d'hygiène de projet, en 6 phases :



1. **Compréhension métier** : c'est quoi le vrai problème ? Quelle décision le modèle doit-il aider à prendre ? (Exemple : "réduire le churn", pas "faire du ML".)
2. **Compréhension des données** : quelles données a-t-on ? Sont-elles fiables, complètes, suffisantes ?
3. **Préparation des données** : nettoyer, corriger, transformer. C'est l'étape la plus longue (souvent 80% du temps). C'est tout le J2.
4. **Modélisation** : entraîner des algos. C'est l'étape la plus visible, mais pas la plus longue.
5. **Évaluation** : le modèle répond-il vraiment au besoin métier de la phase 1 ? Avec les bonnes métriques. C'est le J4.
6. **Déploiement** : mettre le modèle en production pour qu'il serve. Aussi en J4.

Le détail important : ces phases ne sont pas une ligne droite. On boucle. On découvre en phase 3 que la donnée est pourrie, donc on retourne en phase 2 ou même 1. C'est normal et sain.

Remplacez les phases : voici 6 actions d'un projet "détecter les fraudes bancaires", données dans le désordre. Si vous les remettez dans l'ordre CRISP-DM qu'est-ce que ça donne ?

- entraîner un Random Forest sur les transactions
- définir avec la banque ce qu'est une fraude "coûteuse" à rater
- brancher le modèle sur le flux de transactions en temps réel
- repérer que 30% des montants sont manquants et les corriger
- vérifier le taux de fraudes ratées sur des cas réels
- explorer l'historique des transactions disponibles

Approfondissons

CRISP-DM date de 1999 et reste la méthode la plus citée, mais ce n'est pas la seule. Vous croiserez aussi des approches plus orientées équipe et itération courte (inspirées de l'agilité). L'important n'est pas le nom : c'est le réflexe de toujours partir du besoin métier et de toujours finir par "est-ce que ça sert vraiment ?". Un modèle à 99% d'accuracy qui ne répond pas à la question métier est un échec. On reverra ça en J4 avec un exemple où 99% d'accuracy est en fait une catastrophe.

Où ça casse, phase par phase

La théorie des 6 phases, c'est joli. Ce qui est utile, c'est de savoir comment chaque phase plante en vrai, parce que c'est là que les projets meurent.

- **Compréhension métier.** L'échec typique : on n'a jamais vraiment défini le succès. "Faire du ML sur nos clients" n'est pas un objectif. "Réduire de 5% le taux de résiliation en ciblant les clients à risque" en est un. Sans ça, impossible de dire si le projet a réussi.
- **Compréhension des données.** L'échec : on découvre trop tard que la donnée n'existe pas, ou qu'elle est inutilisable. Exemple vécu partout : on veut prédire la fraude, mais personne n'a jamais étiqueté proprement les anciennes fraudes. Pas d'étiquettes, pas de supervisé.
- **Préparation des données.** L'échec : la fameuse fuite de données (data leakage), où une info du futur se glisse dans l'entraînement et donne un score magnifique... qui s'effondre en production. On le démonte en J2.
- **Modélisation.** L'échec : passer trois semaines à optimiser le modèle alors que le vrai gain était dans la donnée. 80% du résultat vient souvent de la préparation, pas de l'algo.
- **Évaluation.** L'échec : se féliciter d'une accuracy élevée sans regarder la bonne métrique. C'est tout le piège des données déséquilibrées (J4).
- **Déploiement.** L'échec : le modèle marche dans le notebook, mais personne ne sait le mettre en production, ou il n'est jamais surveillé et se dégrade en silence (les données du monde réel changent, le modèle vieillit). C'est tout le métier du MLOps.

C'est quoi le MLOps ? Le "DevOps du ML" : tout ce qui permet de mettre un modèle en production, de le surveiller et de le réentraîner quand il se dégrade. Un modèle, contrairement à du code classique, pourrit avec le temps parce que le monde change (on appelle ça le "model drift"). On touche au déploiement en J4.

Les autres méthodes

Pour la culture (et pour ne pas être surpris en réunion) :

- **KDD (Knowledge Discovery in Databases)** : l'ancêtre académique de CRISP-DM, même esprit.
- **SEMMA** : la version de l'éditeur SAS (Sample, Explore, Modify, Model, Assess).
- **TDSP (Team Data Science Process)** : la version de Microsoft, plus orientée travail d'équipe et outillage.

Toutes racontent la même histoire : comprendre, préparer, modéliser, évaluer, déployer, et boucler. Retenez l'esprit, pas l'acronyme.

On a une carte et une méthode. Reste à comprendre la grande division du ML : tous les algos n'apprennent pas de la même façon. C'est la distinction qui structure tout le reste de la semaine.

5 - Les familles d'apprentissage

C'est LA distinction fondamentale du ML, celle qui revient dans tous les entretiens : selon que vos données ont des "réponses" ou non, vous n'êtes pas dans le même monde.

L'essentiel

Apprentissage supervisé : le modèle apprend à partir de **données étiquetées**. Chaque exemple est fourni avec sa réponse attendue. On "supervise" l'apprentissage en montrant les bonnes réponses. Deux grands cas :

- **Classification** : prédire une catégorie. Un email est "spam" ou "non-spam". Une tumeur est "bénigne" ou "maligne". Les étiquettes existent, le modèle apprend à les reproduire.
- **Régression** : prédire une valeur continue. Estimer le prix d'un bien immobilier à partir de sa surface, son emplacement, son année. La réponse est un nombre, pas une catégorie.

En gros, en supervisé, on mâche le travail au modèle : on lui dit "cet exemple, c'est telle réponse", des milliers de fois, jusqu'à ce qu'il généralise.

Apprentissage non-supervisé : cette fois, aucune étiquette. Le modèle cherche une structure dans les données sans qu'on lui dise quoi chercher. Deux grands cas :

- **Clustering (regroupement)** : grouper des clients en segments ("les gros acheteurs nocturnes", "les occasionnels de saison") pour personnaliser le marketing. Personne n'a défini ces groupes à l'avance, l'algo les fait émerger.
- **Réduction de dimension** : transformer 500 colonnes en 2 pour visualiser sur un plan et repérer des motifs invisibles à l'oeil nu. (On verra ça en J2 avec la PCA.)

Donc en non-supervisé, le modèle doit trouver tout seul des motifs cachés, sans guide.

Instant Notions : les deux autres familles

- **Semi-supervisé** : un mélange. On a beaucoup de données, mais seule une petite partie est étiquetée (étiqueter coûte cher). L'algo exploite les deux.

- **Apprentissage par renforcement** : un agent apprend par essai-erreur, en recevant des récompenses ou des punitions. C'est ce qui fait jouer une IA aux échecs ou conduire une voiture en simulation. Hors programme pour nous, mais c'est le troisième grand paradigme, à connaître de nom.

À chaque famille ses algos. Voici la carte qu'on va remplir cette semaine. Pas besoin de tout retenir maintenant, c'est un aperçu :

Famille	Tâche	Algos qu'on verra (J3)
Supervisé	Régression	régression linéaire, polynomiale, Ridge/LASSO
Supervisé	Classification	régression logistique, arbres de décision, Naive Bayes, Random Forest, Gradient Boosting, SVM
Non-supervisé	Clustering	KMeans, classification hiérarchique
Non-supervisé	Réduction de dimension	PCA (J2)

À vous de jouer : pour ces 5 problèmes, je vous laisse voir si c'est supervisé ou non-supervisé, et si supervisé, classification ou régression :

- prédire combien de vélos seront loués demain dans une ville
- regrouper les articles d'un site en thématiques sans liste prédéfinie
- décider si un avis client est positif ou négatif
- estimer la durée de vie restante d'une machine industrielle
- repérer des groupes naturels de joueurs selon leur comportement

Réponses

Problème	Famille	Type
Prédire combien de vélos seront loués demain	Supervisé	Régression
Regrouper les articles en thématiques sans liste prédéfinie	Non-supervisé	Clustering
Décider si un avis client est positif ou négatif	Supervisé	Classification
Estimer la durée de vie restante d'une machine	Supervisé	Régression
Repérer des groupes naturels de joueurs	Non-supervisé	Clustering

On va plus loin

Une subtilité utile : un même problème métier peut basculer d'une famille à l'autre selon la donnée disponible. "Segmenter mes clients" est non-supervisé si je n'ai aucun groupe prédéfini. Mais si le marketing me fournit déjà des catégories sur un échantillon, ça devient de la classification supervisée. La famille n'est pas une propriété du problème, c'est une propriété de la **donnée que vous avez**. Voilà pourquoi la phase

"compréhension des données" de CRISP-DM est si décisive !

Au-delà des deux grandes familles

Supervisé et non-supervisé couvrent l'essentiel, mais le paysage est plus riche. Pour la culture (et pour ne pas bloquer quand on vous balance ces mots) :

- **Apprentissage semi-supervisé.** On a une montagne de données, mais étiqueter coûte cher (faire annoter un million d'images par des humains, c'est un budget). Alors on étiquette un petit bout, et l'algo se sert aussi du reste, non étiqueté, pour s'améliorer. Très courant en vision et en NLP.
- **Apprentissage auto-supervisé (self-supervised).** La grande idée derrière les modèles modernes, ChatGPT compris. On fabrique les étiquettes automatiquement à partir de la donnée elle-même : "cache un mot dans une phrase et demande au modèle de le deviner". Pas besoin d'humain pour étiqueter, et il y a du texte à l'infini sur le web. C'est ce qui a rendu les LLM possibles.
- **Apprentissage par renforcement (RL).** Un agent apprend par essai-erreur dans un environnement, guidé par des récompenses. Pas d'étiquettes : juste "ce que tu viens de faire t'a rapporté +1 ou -1". C'est ce qui fait jouer AlphaGo, pilote des robots, ou optimise des stratégies.

C'est quoi le RLHF ? "Reinforcement Learning from Human Feedback". On entraîne un modèle déjà bon en langage à préférer les réponses que des humains jugent meilleures. C'est l'ingrédient secret qui transforme un modèle qui "complète du texte" en assistant qui "répond correctement". La touche finale de ChatGPT et autres.

Tout ça est hors programme pour nous : on reste sur le supervisé et le non-supervisé, qui couvrent l'immense majorité des besoins en entreprise. Mais maintenant, ces mots ne vous feront plus peur.

La carte complète des algos

On va remplir cette carte toute la semaine. La voici en entier, une phrase par algo, pour que vous ayez la vue d'ensemble avant de plonger. Pas de panique, on les reverra un par un en J3.

Supervisé, régression (prédire un nombre) :

- **Régression linéaire** : trace la meilleure droite à travers les points. Simple, rapide, explicable.
- **Régression polynomiale** : une courbe au lieu d'une droite, pour les relations non linéaires.
- **Ridge / LASSO** : des régressions linéaires "bridées" pour éviter le surapprentissage (LASSO peut même supprimer les features inutiles).

Supervisé, classification (prédire une catégorie) :

- **Régression logistique** : malgré son nom, c'est de la classification. Sort une probabilité. La base du scoring.
- **K plus proches voisins (KNN)** : "dis-moi qui sont tes voisins, je te dirai qui tu es". Pas d'entraînement, on compare aux exemples connus.
- **Arbre de décision** : une suite de questions oui/non. Très lisible, mais fragile tout seul.
- **Naive Bayes** : applique la loi de Bayes en supposant les features indépendantes. Imbattable en vitesse sur le texte (spam).
- **Random Forest** : une forêt d'arbres qui votent. Robuste, peu de réglages, excellent par défaut.

- **Gradient Boosting (XGBoost, LightGBM)** : des arbres en séquence qui corrigent les erreurs des précédents. Le roi du tabulaire en compétition.
- **SVM (machine à vecteurs de support)** : trace la frontière qui sépare le mieux les classes. Puissant sur des données complexes mais de taille raisonnable.

Non-supervisé :

- **KMeans** : regroupe en k clusters autour de centres. Le clustering le plus courant.
- **Classification hiérarchique** : construit un arbre de regroupements, sans fixer k à l'avance.
- **PCA** : réduit le nombre de colonnes en gardant l'essentiel de l'information (J2).

Aparté : ne vous noyez pas dans cette liste. La vraie compétence, ce n'est pas de connaître douze algos par coeur, c'est de savoir lequel essayer en premier et comment les comparer proprement. C'est tout l'objet de l'Arène.

Apprentissage en ligne ou par lots ?

Une dernière distinction qui revient en entreprise :

- **Par lots (batch)**. On entraîne le modèle d'un coup sur tout l'historique, on le fige, on l'utilise. C'est ce qu'on fait cette semaine. Simple et largement suffisant.
- **En ligne (online)**. Le modèle se met à jour en continu, exemple après exemple, au fil de l'arrivée des données. Utile quand le monde change vite (fraude, pub en temps réel) ou que la donnée ne tient pas en mémoire.

Pour démarrer, le batch suffit. Sachez juste que le online existe, quand on vous parlera de "streaming".

On sait quels types d'apprentissage existent et quels algos vont avec. Maintenant, avec quels outils on code tout ça ? Petit tour du marché, puis on choisit le nôtre.

6 - Les outils : panorama des frameworks

Vous lisez une offre d'emploi data. Elle demande "scikit-learn", ou "PyTorch", ou "TensorFlow/Keras". Pour la moitié des recruteurs ces termes sont interchangeables. Pour vous, après cette section, ils ne le seront plus.

L'essentiel

Il faut distinguer deux mondes d'outils, parce qu'ils ne servent pas à la même chose.

Pour le ML classique (notre coeur, J1 à J3) : [scikit-learn](#). C'est LA bibliothèque Python du ML classique. Régressions, arbres, forêts, SVM, clustering, métriques, préparation des données : tout y est, avec une interface unique et limpide. C'est l'outil central de ce cours. Si vous ne deviez en maîtriser qu'un cette semaine, c'est celui-là.

Pour le Deep Learning (on l'ouvre en J4) : les frameworks de réseaux de neurones. Ils résolvent un autre problème : faire des calculs sur GPU et calculer automatiquement les dérivées pour entraîner des réseaux. Les principaux, pour culture :

Framework	Par qui	En un mot
TensorFlow / Keras	Google	Le combo grand public, fort en production. On l'utilise un peu en J4.
PyTorch	Meta	Le préféré de la recherche, très "pythonique".
Theano, Torch, Caffe	(historiques)	Les ancêtres, que vous croiserez dans de vieux articles. Theano (2010) est le grand-père, arrêté en 2017 ; Caffe était la référence vision avant PyTorch.

Si on devait imaginer tout ça : Keras est le volant, TensorFlow est le moteur. Keras est une interface simple posée au-dessus d'un moteur de calcul. On choisira Keras en J4 parce que la courbe d'apprentissage est courte : en une demi-journée de découverte, on veut comprendre les concepts, pas batailler avec la syntaxe.

Deux outils compagnons qu'on utilisera en permanence, à connaître :

Instant Cheat Sheet! : la trousse de base

- [pandas](#) : manipuler des tableaux de données (charger un CSV, filtrer, calculer des moyennes). Le couteau suisse de la donnée tabulaire.
- [numpy](#) : le calcul numérique sur des tableaux de nombres. La fondation sur laquelle pandas et scikit-learn sont bâtis.
- [matplotlib](#) : tracer des courbes et des graphiques. Pour voir ce qu'on fait.

`import sklearn` : dans un notebook, exécutez ceci :

```
from sklearn.datasets import load_iris

# load_iris : un jeu de données jouet livré avec scikit-learn
# 150 fleurs d'iris, 4 mesures par fleur, 3 espèces à distinguer
X, y = load_iris(return_X_y=True)

print("Forme des données (lignes, colonnes) :", X.shape)
print("Premières mesures :", X[0])
print("Espèces possibles :", set(y))
```

Devrait afficher :

```
Forme des données (lignes, colonnes) : (150, 4)
Premières mesures : [5.1 3.5 1.4 0.2]
Espèces possibles : {0, 1, 2}
```

Essayez : remplacez `load_iris` par `load_wine`, relancez, et regardez comment la forme change. Combien de colonnes pour le vin ? Comparez avec le voisin.

Pour démarrer (les Getting Started officiels) : chacun de ces outils a évidemment sa propre page de démarrage pensée pour ceux qui démarrent. Ça peut partir en favori, je vous invite à les consulter dès maintenant.

- scikit-learn : [Getting Started](#)

- pandas : [10 minutes to pandas](#)
- numpy : [NumPy, les bases absolues](#)
- matplotlib : [Quick start guide](#)
- Keras : [Getting started](#) (utile pour J4)
- PyTorch : [Learn the Basics](#) (pour la culture)
- Google Colab : [notebook d'introduction](#)
- Kaggle : [Kaggle Learn](#), des micro-cours gratuits pour débiter
- HuggingFace : [HuggingFace Learn](#), les cours maison pour aller vers le DL et le NLP

Pour les curieux

Attendez, je n'ai toujours pas mentionné cette pépite ?!

[HuggingFace](#) c'est clairement le GitHub de l'IA => la plateforme de référence pour les modèles de langage pré-entraînés, type ChatGPT,

FastAI et **PyTorch Lightning** (des surcouches qui structurent PyTorch). Aucun n'est nécessaire pour ce cours.

Un conseil, le réflexe pro à retenir : choisir l'outil le plus simple qui résout le problème. Pour 90% des problèmes tabulaires d'entreprise, c'est scikit-learn, et un réseau de neurones serait du gâchis.

Un dernier mot, sur l'open source, parce que c'est le coeur de l'écosystème : scikit-learn, pandas, XGBoost, HuggingFace... tout ça est gratuit, ouvert, et maintenu par des communautés. Vous ne payez aucune licence, vous pouvez lire le code source quand un truc vous intrigue, et même contribuer. Une correction de doc acceptée sur scikit-learn, c'est une vraie ligne de CV : ça prouve que vous savez lire du code pro et collaborer. On code tous sur les épaules de géants (souvent bénévoles).

Pour creuser : les dépôts de [scikit-learn](#) et [pandas](#) sur GitHub.

L'écosystème autour

scikit-learn ne vit pas seul. Quelques noms qui reviennent et qui valent un repère, même si on ne les utilise pas tous cette semaine :

- **XGBoost, LightGBM, CatBoost** : les bibliothèques de gradient boosting, les rois du tabulaire en compétition. Elles s'utilisent presque comme scikit-learn. On croise XGBoost en J3.
- **imbalanced-learn** : l'extension de scikit-learn pour gérer les données déséquilibrées (la fraude, encore). Utile dès qu'une classe est rare.
- **Optuna** : pour chercher automatiquement les meilleurs hyperparamètres, au lieu de tâtonner à la main. On en reparle en J4.
- **MLflow, Weights & Biases** : pour suivre ses expériences (quel réglage a donné quel score ?). Indispensable dès qu'on lance des dizaines d'entraînements.
- **ONNX** : un format universel pour exporter un modèle et le faire tourner n'importe où (mobile, navigateur, autre langage). Utile au déploiement.

Vous n'avez besoin d'aucun de ces outils aujourd'hui. Mais quand vous lirez du code pro ou une offre d'emploi, ces noms ne seront plus du chinois.

Notebooks, environnements et GPU

Le décor technique, vite fait, pour que vous soyez à l'aise :

- **Notebook (Jupyter, Colab)** : un document qui mélange code, résultats et texte, cellule par cellule. Parfait pour explorer et apprendre. En production, on repasse souvent à des scripts `.py` classiques.
- **Environnement virtuel (venv, conda)** : une bulle isolée avec les bonnes versions de bibliothèques pour un projet, pour éviter que deux projets se marchent dessus. Bonne pratique pro dès qu'on installe des paquets.
- **CPU contre GPU**. Le ML classique (notre semaine, sauf J4) tourne très bien sur le processeur normal (CPU) de votre machine. Le Deep Learning, lui, a besoin d'une carte graphique (GPU) pour aller vite. Bonne nouvelle : Google Colab vous en prête une gratuitement, ce qui nous sauvera en J4.

Bon à savoir : pour tout ce qu'on fait en J1-J3, pas besoin de GPU ni de machine de guerre. Un Colab gratuit, ou même votre laptop, suffisent largement. Le ML classique est frugal, c'est un de ses gros avantages.

Pourquoi Python a gagné

Vous pourriez faire du ML en R, en Julia, voire en Java. Mais le monde a convergé vers Python, pour de bonnes raisons : un langage simple à lire, un écosystème scientifique énorme (numpy, pandas, scikit-learn...), et une communauté gigantesque. Résultat : la quasi-totalité des tutos, des offres d'emploi et des modèles partagés sont en Python. R reste fort en stats académiques, Julia monte côté calcul scientifique, mais pour un job data aujourd'hui, Python est le pari sûr. Tant mieux : c'est celui que vous connaissez déjà.

Okay on a les outils en main et on vient de charger nos premières données. Le moment est venu de faire ce pour quoi on est là : entraîner un modèle et lui faire prédire quelque chose.

7 - Votre premier pipeline : charger, entraîner, prédire, mesurer

Tout ce qu'on a vu converge ici. Un "pipeline" de ML, c'est l'enchaînement de bout en bout : des données brutes jusqu'à une prédiction mesurée. Et la beauté de scikit-learn, c'est que cet enchaînement est toujours le même, quel que soit l'algo.

L'essentiel

Voici un pipeline complet et fonctionnel. Lisez les commentaires : ils expliquent le "pourquoi" de chaque étape.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Charger les données : 150 fleurs, 4 mesures chacune, 3 espèces
X, y = load_iris(return_X_y=True)

# 2. Séparer apprentissage et test.
```

```
# Pourquoi ? On juge un modèle sur des données qu'il n'a jamais vues,
# sinon c'est comme noter un élève sur les exercices qu'il avait en révision.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Choisir un modèle et l'entraîner (.fit = "apprends sur ces données")
modele = DecisionTreeClassifier(random_state=42)
modele.fit(X_train, y_train)

# 4. Prédire sur des données jamais vues (.predict)
y_pred = modele.predict(X_test)

# 5. Mesurer : quelle proportion de bonnes réponses ?
print(f"Accuracy : {accuracy_score(y_test, y_pred):.2%}")
```

Devrait afficher quelque chose comme :

```
Accuracy : 96.67%
```

Cinq étapes, et c'est tout. Charger, séparer, entraîner, prédire, mesurer. Retenez ce squelette : c'est le coeur de tout ce qu'on fera cette semaine.

C'est quoi `random_state` ? Une graine de hasard fixée. Le découpage train/test et certains algos utilisent de l'aléatoire ; en fixant `random_state`, on obtient le même résultat à chaque exécution, ce qui rend les comparaisons justes et reproductibles.

Et `stratify=y` ? Ça garantit que les proportions de chaque espèce sont les mêmes dans l'apprentissage et dans le test. Sans ça, le hasard pourrait mettre presque toutes les fleurs d'une espèce du même côté.

Le point qui change tout : l'API uniforme. Dans scikit-learn, presque tous les modèles s'utilisent pareil : `.fit()` pour apprendre, `.predict()` pour prédire, `.score()` pour mesurer. Changer d'algo, c'est changer une seule ligne.

Manip express : dans le code ci-dessus, remplacez la ligne du modèle par une régression logistique, puis par un autre algo, et comparez l'accuracy. Vous ne touchez QUE la ligne du modèle (et l'import) :

```
from sklearn.linear_model import LogisticRegression
modele = LogisticRegression(max_iter=200)
```

puis :

```
from sklearn.neighbors import KNeighborsClassifier
modele = KNeighborsClassifier(n_neighbors=3)
```

Lequel gagne sur l'iris ? Notez les trois scores. Bienvenue dans l'Arène : vous venez de faire votre premier mini-classement d'algos. Comparez votre podium avec le voisin.

Pour aller plus loin

Vous avez peut-être remarqué que les scores varient un peu selon l'algo, mais tous sont très bons : l'iris est un dataset facile. La vraie difficulté en entreprise, ce n'est pas de faire tourner `.fit()`, c'est que les données sont sales, déséquilibrées, pleines de trous. Un modèle entraîné sur une donnée pourrie prédit n'importe quoi, peu importe sa sophistication. C'est tout l'enjeu de demain. Gardez aussi en tête une question qu'on creusera : l'accuracy suffit-elle pour juger un modèle ? Spoiler : non, et on verra pourquoi en J4.

Train, validation, test : pourquoi parfois trois jeux

On a coupé en deux (train / test). En vrai, les pros coupent souvent en trois, et c'est important de comprendre pourquoi :

- **Train (entraînement)** : le modèle apprend dessus.
- **Validation** : on s'en sert pour régler les hyperparamètres et choisir entre plusieurs modèles, sans toucher au test.
- **Test** : on n'y touche qu'à la toute fin, une seule fois, pour estimer la vraie performance. Si on règle nos choix sur le test, on triche sans le savoir : on finit par s'adapter à lui, et notre score devient trop optimiste.

L'image : le train est le cours, la validation les exercices d'entraînement, le test l'examen final. Réviser sur le sujet exact de l'examen, c'est tricher.

C'est quoi la validation croisée (cross-validation) ? Plutôt que de figer un seul découpage, on coupe les données en k morceaux, on entraîne k fois en gardant à chaque fois un morceau différent pour valider, et on moyenne. Ça donne une estimation bien plus fiable, surtout sur petit dataset. On la pratique en J4. Retenez le mot, il va beaucoup revenir.

L'objet Pipeline de scikit-learn

Le mot "pipeline" a deux sens, attention. Il y a le pipeline-concept (les 5 étapes). Et il y a l'objet `Pipeline` de scikit-learn, qui empaquette préparation et modèle dans un seul bloc :

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# La mise à l'échelle ET le modèle réunis dans un seul objet
modele = make_pipeline(StandardScaler(), LogisticRegression())
modele.fit(X_train, y_train)    # le scaler s'ajuste sur le train, puis le modèle
                                # s'entraîne
modele.predict(X_test)         # le test est mis à l'échelle avec les MÊMES réglages,
                                # automatiquement
```

L'intérêt n'est pas cosmétique : c'est le meilleur garde-fou contre la fuite de données. Le scaler s'ajuste sur le train seulement, et s'applique à l'identique au test, sans que vous ayez à y penser. On s'en resservira beaucoup à partir de J2.

Les hyperparamètres, le vrai levier

Vous avez écrit `KNeighborsClassifier(n_neighbors=3)`. Ce `3`, c'est un hyperparamètre : un réglage que VOUS choisissez. Changez-le pour 5, 10, 20, et l'accuracy bouge.

- Un arbre trop profond apprend par coeur (overfitting) ; trop court, il rate des motifs (underfitting).
- Une forêt avec 10 arbres est moins stable qu'avec 300.
- Le `c` d'une SVM contrôle à quel point elle tolère des erreurs d'entraînement.

Trouver les bons réglages, ça s'appelle le "tuning". On peut le faire à la main (comme cet après-midi), ou automatiquement avec une recherche (GridSearch, ou Optuna). On le formalise en J4. Le réflexe à prendre dès maintenant : un même algo peut être nul ou excellent selon ses hyperparamètres. Ne jugez jamais un algo sur un seul réglage.

8 - Limites, biais et responsabilité

On a vendu du rêve toute la matinée. Trente secondes de lucidité maintenant, parce que c'est ce qui sépare un bricoleur d'un pro : un modèle de ML n'est ni neutre, ni infaillible.

L'essentiel

Un modèle apprend de ses données. Si les données sont biaisées, le modèle l'est aussi, et il automatise le biais à grande échelle. Trois cas réels célèbres :

- **Boston Housing.** Un jeu de données immobilier longtemps utilisé pour apprendre le ML... jusqu'à ce qu'on réalise qu'il contenait une variable bâtie sur des critères racistes. Retiré de scikit-learn pour ça. (On utilisera California Housing à la place en J3.)
- **COMPAS.** Un outil américain qui prédisait le risque de récidive. Une enquête a montré qu'il surévaluait le risque pour les personnes noires. Et il servait à éclairer des décisions de justice.
- **L'outil de recrutement d'Amazon.** Entraîné sur dix ans de CV majoritairement masculins, il s'est mis à pénaliser les CV qui contenaient des indices féminins. Abandonné.

La leçon : ce ne sont pas les algos qui sont "méchants", ce sont les données qui portent les biais du monde réel, et le modèle les amplifie sans état d'âme.

C'est quoi le "droit à l'explication" ? Avec le RGPD, une personne peut exiger de comprendre une décision automatisée qui la concerne (un crédit refusé, par exemple). Conséquence très concrète : en banque ou en assurance, on préfère souvent un modèle explicable (régression, arbre) à une boîte noire plus performante mais opaque. La performance n'est pas le seul critère.

C'est aussi pour ça que le J2 (comprendre et nettoyer la donnée) et le J4 (bien évaluer) ne sont pas des détails de confort : c'est là qu'on attrape les biais avant qu'ils ne fassent des dégâts.

Creusons

Le piège classique : "mes données sont objectives, ce sont des chiffres". Faux. Quelqu'un a choisi quoi mesurer, sur qui, et comment étiqueter. Un dataset est toujours le reflet de choix humains. Le réflexe pro avant d'entraîner quoi que ce soit : se demander "qui est sur-représenté ? qui manque ? qu'est-ce que la cible mesure vraiment ?". On remettra ces lunettes en J2.

D'autres cas qui ont marqué

Trois cas de plus, parce que le sujet est trop important pour le survoler :

- **Google Photos (2015).** L'algo de reconnaissance d'images a étiqueté des personnes noires comme des "gorilles". Cause : un jeu d'entraînement qui manquait cruellement de diversité. La "solution" de Google a longtemps été de... supprimer le mot-clé gorille. Symbole du problème : un modèle ne voit que ce qu'on lui a montré.
- **Un algorithme de santé américain (2019, revue Science).** Un outil utilisé sur des millions de patients orientait moins de soins vers les patients noirs. Pas par méchanceté : il utilisait les "dépenses de santé passées" comme proxy du besoin de soins. Or, à besoin égal, on dépensait historiquement moins pour ces patients. Le proxy portait le biais.
- **La reconnaissance faciale.** Plusieurs études (dont *Gender Shades* de Joy Buolamwini) ont montré des taux d'erreur bien plus élevés sur les femmes à peau foncée que sur les hommes à peau claire. Avec des conséquences réelles : des arrestations sur de fausses identifications.

Le point commun : aucun de ces systèmes n'était "méchant". Chacun a fidèlement appris ce que sa donnée contenait. C'est ça le piège, et c'est pour ça que comprendre sa donnée (J2) est une responsabilité, pas juste une étape technique.

Les types de biais

Savoir les nommer aide à les traquer :

- **Biais de sélection** : la donnée ne représente pas la réalité visée (on entraîne sur les clients actuels pour prédire sur de futurs clients différents).
- **Biais de mesure** : on mesure mal, ou on utilise un proxy trompeur (les "dépenses de santé" pour le "besoin de soins").
- **Biais d'étiquetage** : les étiquettes elles-mêmes portent les préjugés de ceux qui les ont posées.
- **Biais historique** : la donnée reflète fidèlement un monde déjà inégal, et le modèle perpétue l'inégalité (l'outil de recrutement d'Amazon).

Le réflexe à garder : "garbage in, garbage out". Un modèle ne peut pas être plus juste que sa donnée. Avant de blâmer l'algo, regardez ce qu'on lui a donné à manger.

La régulation et le coût caché

Deux sujets qui montent et qu'un pro doit avoir en tête :

- **La régulation.** Au-delà du RGPD (droit à l'explication), l'Union européenne a voté l'**AI Act**, qui classe les usages de l'IA par niveau de risque et impose des obligations sur les usages sensibles (recrutement, crédit, justice). Concrètement : de plus en plus, un modèle qui décide pour des humains devra être explicable et auditable. L'explicabilité (avec des outils comme SHAP ou LIME, qui montrent quelles features ont pesé sur une décision) devient une compétence recherchée.
- **Le coût environnemental.** Entraîner un gros modèle de Deep Learning consomme énormément d'électricité (certains entraînements de LLM se chiffrent en centaines de tonnes de CO2). Encore un argument pour le ML classique : un Random Forest sur un laptop, c'est quelques secondes et quasi rien. Le bon modèle n'est pas toujours le plus gros.

Pour creuser : le documentaire *Coded Bias* et le projet [Gender Shades](#) de Joy Buolamwini, pour voir ces enjeux en grand. Pas au programme, mais ça change le regard.

Voilà pour les garde-fous. On garde ces réflexes en tête, et on passe à la pratique.

À votre tour

On a posé les concepts ce matin. Cet après-midi, vous pilotez. L'objectif : monter un pipeline ML complet de bout en bout sur un vrai dataset, et le faire entrer dans l'Arène en comparant plusieurs algos. À la fin, vous avez votre premier repo de la semaine.

Le projet

Vous prenez un dataset de classification, vous le faites passer par le pipeline complet, et vous ouvrez votre Arène : plusieurs algos qui s'affrontent sur un même classement. Ensuite vous élargissez (le non-supervisé, d'autres datasets, des visualisations, l'optimisation du champion), et vous documentez vos choix.

Le livrable : un notebook propre + un README qui présente votre classement et justifie votre champion, poussés sur GitHub.

Vous avez les signatures, les sorties attendues et les étapes. À vous d'écrire le code. Si vous bloquez c'est normal (et même volontaire 🐈) : c'est là qu'on apprend.

Phase 0 : Mise en route (setup)

Avant le code, l'intendance, une bonne fois :

- Ouvrez [Google Colab](#) (rien à installer) ou votre Jupyter local.
- Créez un compte [Kaggle](#) si ce n'est pas fait (on s'en sert pour les datasets dès J2).
- Créez un repo GitHub pour la semaine, par exemple `arene-des-algos-<Prenom_NOM>`. Premier commit : un README qui dit ce que vous allez y mettre.

Avant de coder : votre repo GitHub est votre copie d'examen continue. On note les commits, pas seulement le résultat final. Committez souvent, à chaque phase finie, avec des messages clairs. Pas de `git add .` à l'aveugle : ajoutez les fichiers un par un !

Écrire des **commits atomiques**. (verbe(secteur / scope): on a fait ça)

Exemples :

```
feat(home): add bottom section
fix(auth): add bottom section
style(home): add bottom section
content(home): add bottom section
feat(home): add bottom section
feat(home): add bottom section
feat(#34 home): add bottom section      # avec le numéro du ticket
```

Phase 1 : Charger et explorer

Chargez un dataset de classification fourni par scikit-learn (`load_breast_cancer` est un bon choix : 569 patients, 30 mesures, tumeur bénigne ou maligne). Inspectez-le avant toute chose.

```
def explorer_dataset():
    """Charge le dataset et affiche ses caractéristiques de base.
    Doit afficher : nombre de lignes, nombre de colonnes,
    les classes possibles et leur répartition (équilibrée ou non ?).
    """
    # TODO : charger le dataset (return_X_y=True donne X et y séparés)
    # TODO : afficher la forme de X (combien de lignes, combien de colonnes)
    # TODO : compter et afficher combien d'exemples dans chaque classe
    pass
```

Devrait afficher quelque chose comme :

```
Lignes, colonnes : (569, 30)
Classe 0 (maligne) : 212 cas
Classe 1 (bénigne) : 357 cas
```

Checkpoint qualité : testez votre fonction sur 3 situations et vérifiez qu'elle ne ment pas :

- cas normal : le dataset complet
- cas limite : que se passe-t-il si vous ne gardez qu'une seule classe (filtrez `y == 0`) ? Le décompte est-il honnête ?
- cas adversarial : affichez la répartition en pourcentage. Si une classe faisait 95%, est-ce que votre exploration le rendrait visible ? (C'est crucial pour la suite.)

Phase 2 : Le pipeline supervisé complet

Reprenez les 5 étapes du matin sur ce dataset : split, entraînement, prédiction, mesure. Encapsulez dans une fonction qui prend un modèle en argument.

```
def entrainer_et_evaluer(modele, X_train, X_test, y_train, y_test):
    """Entraîne le modèle, prédit sur le test, renvoie l'accuracy.
    Doit renvoyer un float entre 0 et 1.
    """
    # TODO : entraîner le modèle sur les données d'apprentissage
    # TODO : prédire sur le jeu de test
    # TODO : calculer l'accuracy et la renvoyer
    pass
```

Rappel de méthode : committez ici. Un commit = une étape finie. Message du type `feat: pipeline supervisé sur breast cancer`.

Phase 3 : L'Arène (premier classement)

Faites s'affronter au moins 3 algos sur le MÊME découpage train/test. Affichez un classement trié par accuracy.

```
def arene(X_train, X_test, y_train, y_test):  
    """Entraîne plusieurs modèles, renvoie un classement trié.  
    Doit afficher un tableau lisible : nom de l'algo, accuracy.  
    """  
    # TODO : construire un dictionnaire {nom: modèle} avec au moins 3 algos  
    #         (repensez à ceux du matin : arbre, régression logistique, KNN...)  
    # TODO : pour chaque modèle, réutiliser entraîner_et_evaluer sur le MÊME split  
    # TODO : trier par accuracy décroissante et afficher le podium  
    pass
```

Devrait afficher quelque chose comme :

```
1. Régression logistique : 97.4%  
2. KNN                    : 95.6%  
3. Arbre de décision      : 93.9%
```

Ce qui doit casser (et que vous devez gérer) : la régression logistique peut afficher un avertissement de non-convergence. Pourquoi ? Que se passe-t-il si vous augmentez `max_iter` ? Notez ce que vous observez : c'est un premier indice que certains algos ont besoin de données mises à l'échelle (on verra le scaling en J2).

Phase 4 : Bascule non-supervisé

Maintenant, cachez les étiquettes. Lancez un KMeans qui doit trouver 2 groupes tout seul, sans jamais voir `y`. Puis comparez : les groupes trouvés correspondent-ils aux vraies classes (bénigne/maligne) ?

```
from sklearn.cluster import KMeans  
  
def clustering_aveugle(X):  
    """Regroupe les données en 2 clusters sans les étiquettes.  
    Renvoie les labels de cluster attribués à chaque point.  
    """  
    # TODO : créer un KMeans à 2 clusters et renvoyer le cluster de chaque point  
    #         (indice : une seule méthode fait l'entraînement ET la prédiction d'un coup)  
    pass
```

Question à creuser, sans réponse unique : un cluster non-supervisé qui retrouve à peu près les vraies classes, est-ce de la chance, ou est-ce que la structure était vraiment dans les données ? Comparez le clustering aux vraies étiquettes et discutez-en par deux.

Phase 5 : Changer de terrain

Reprenez toute votre Arène, mais sur un autre dataset. `load_wine` est parfait : 3 classes au lieu de 2 (donc de la classification multi-classe), 13 mesures. Relancez l'exploration puis le classement, sans réécrire vos fonctions.

À vérifier : un algo qui gagnait sur le cancer du sein passe-t-il derrière sur le vin ? Votre fonction `arene` encaisse-t-elle un dataset à 3 classes sans aucune modification (elle le devrait, c'est tout l'intérêt de l'API uniforme) ? Et si une classe est moins représentée que les autres, est-ce que ça se voit dans vos résultats ?

Phase 6 : Voir pour comprendre

Un classement de chiffres, c'est bien. Un graphique, c'est mieux pour décider. Avec matplotlib, tracez un diagramme en barres des accuracies de vos algos. Puis, pour votre champion, affichez sa **matrice de confusion** : le tableau qui montre non seulement combien d'erreurs il fait, mais lesquelles (quelles classes il confond).

C'est quoi une matrice de confusion ? Un tableau qui croise les vraies classes et les classes prédites. Sur la diagonale, les bonnes réponses ; ailleurs, les erreurs, et on voit exactement lesquelles. On la creuse en J3.

Trois choses à regarder : votre graphique est-il lisible sans le code (titres, axes nommés) ? Votre champion se trompe-t-il plus dans un sens que dans l'autre ? Et sur le cancer du sein, qu'est-ce qui est le plus grave, rater une tumeur maligne ou alerter à tort sur une bénigne ? (Reliez ça à la section "limites" du matin.)

Phase 7 : Le buff scaling (et la triche qui se retourne contre vous)

Souvenez-vous de l'avertissement de la régression logistique en Phase 3. Certains algos sont aveugles aux échelles, d'autres en souffrent énormément. On va le prouver chiffres en main, puis tomber exprès dans un piège que presque tout le monde se prend une fois dans sa carrière.

Manche 1 : le buff. Mettez vos données à l'échelle avec un `StandardScaler` (il ramène chaque colonne sur une moyenne de 0 et un écart-type de 1, donc une échelle comparable), et relancez l'Arène complète. Pour chaque algo, mesurez le gain : accuracy scalée moins accuracy brute.

```
def comparer_scaling(modeles, X_train, X_test, y_train, y_test):
    """Pour chaque algo, mesure l'accuracy SANS puis AVEC scaling.
    Doit afficher un tableau trié par gain décroissant : nom, brut, scalé, delta.
    """
    # TODO : pour chaque modèle, évaluer une première fois sur les données brutes
    # TODO : ajuster un StandardScaler sur le TRAIN seul, transformer train ET test
    # TODO : ré-évaluer le même modèle sur les données scalées
    # TODO : calculer le delta (scalé - brut), trier par delta décroissant, afficher
    pass
```

Devrait afficher quelque chose comme :

Algo	Brut	Scalé	Gain
Régression logistique	94.7%	98.2%	+3.5
KNN	95.6%	97.4%	+1.8
Arbre de décision	93.9%	93.9%	+0.0

Le classement des gains EST le résultat de cette manche, pas l'accuracy brute. Qui profite du buff, qui n'en a rien à faire, et est-ce que ça colle à votre intuition du matin ? (Indice : un algo qui raisonne par distances n'a pas le même rapport aux échelles qu'un algo qui découpe par seuils.)

Manche 2 : la triche. Maintenant on triche volontairement. Ajustez votre scaler sur le dataset COMPLET (`fit` sur tout X, AVANT le split), puis splittez et relancez votre champion. Notez sa nouvelle accuracy.

C'est quoi le souci ? Vous venez de laisser le scaler « regarder » les données de test pour calculer sa moyenne et son écart-type. Le modèle bénéficie donc d'une info qu'il n'aurait jamais en vrai : c'est une fuite de données (data leakage). L'accuracy monte... mais elle ment. On formalise tout ça en J2.

Le verdict qui pique : combien de points votre champion a-t-il gagnés EN TRICHANT (scaler ajusté sur tout) par rapport à la version honnête (scaler ajusté sur le train seul) ? Ce delta, c'est exactement le mensonge que vous vous raconteriez à vous-même en prod. Sur ce dataset il sera peut-être minuscule ; sur d'autres il transforme un modèle moyen en faux champion qui s'effondre le jour du déploiement. Le réflexe à graver à vie : on ajuste TOUJOURS le scaler sur le train seul. Et le `make_pipeline` croisé ce matin existe précisément pour rendre cette triche impossible, même par accident.

Pensez à committer : une manche finie = un commit. Message du type `feat: comparatif scaling + démo data leakage`.

Phase 8 : Raconter votre Arène

Un résultat que personne ne comprend ne sert à rien. Complétez le README de votre repo avec une **note de synthèse** : le problème, les algos comparés, votre tableau de classement final (tous datasets confondus), le champion retenu, et surtout **pourquoi** vous le choisiriez en vrai. Pas seulement l'accuracy : pensez explicabilité, type d'erreurs, vitesse. Écrivez pour quelqu'un qui ne code pas.

Le test ultime : faites lire votre README à votre binôme. En une minute, a-t-il compris quel algo vous recommandez et pourquoi ? Si non, ce n'est pas son attention qu'il faut blâmer, c'est le README qu'il faut reprendre.

Peer review (relecture par deux)

Avant de committer la version finale, échangez votre notebook avec un binôme. Vous vous relisez mutuellement avec cette grille (ce n'est pas une note, c'est une discussion) :

- Le pipeline suit-il bien les 5 étapes, dans l'ordre ?
- Le train et le test sont-ils bien séparés (pas de fuite) ?
- Le classement est-il lisible par quelqu'un qui n'a pas écrit le code ?
- Y a-t-il des choix différents des vôtres qui sont tout aussi valables ? Lesquels ?

Le but : voir qu'il n'y a pas une seule bonne réponse. Le choix du dataset, des algos, de la métrique, tout ça se discute.

Attention !

Si votre pipeline d'aujourd'hui n'est pas committé sur GitHub, faites-le : c'est la trace de votre travail. **Il me faut votre repo dès ce milieu de journée !!**

Récapitulatif Jour 1

Ce qu'on a appris aujourd'hui

1. **Le paysage** : Data Science englobe l'IA, qui englobe le ML, qui englobe le DL. Le ML, c'est apprendre par l'exemple, et il vit à l'étape "modélisation" de la chaîne de la donnée.
2. **Un peu d'histoire** : du Perceptron (1958) à ChatGPT (2022), et pourquoi forêts et boosting règnent encore sur le tabulaire.
3. **Les cas d'usage** : banque, assurance, télécom, santé, NLP, jeux. Et la culture des challenges avec Kaggle.
4. **CRISP-DM** : les 6 phases d'un projet data, et le réflexe de toujours partir du besoin métier.
5. **Les familles d'apprentissage** : supervisé (classification, régression) vs non-supervisé (clustering, réduction de dimension), plus le semi-supervisé et le renforcement.
6. **Les outils** : scikit-learn au centre pour le ML classique, les frameworks DL (Keras/TensorFlow, PyTorch) pour plus tard.
7. **Le pipeline** : charger, séparer, entraîner, prédire, mesurer. L'API uniforme `.fit` / `.predict` / `.score`.
8. **Limites et biais** : un modèle apprend les biais de ses données. Données propres et bonne évaluation ne sont pas optionnelles.

Points d'attention pour demain

- Demain, on attaque le nerf de la guerre : la **préparation de la donnée**. Vous avez vu que la régression logistique râlait sans mise à l'échelle ? On comprendra pourquoi, et on apprendra à nettoyer une donnée sale (trous, valeurs aberrantes, doublons).
- Gardez en tête la question laissée ouverte : l'accuracy suffit-elle vraiment à juger un modèle ?